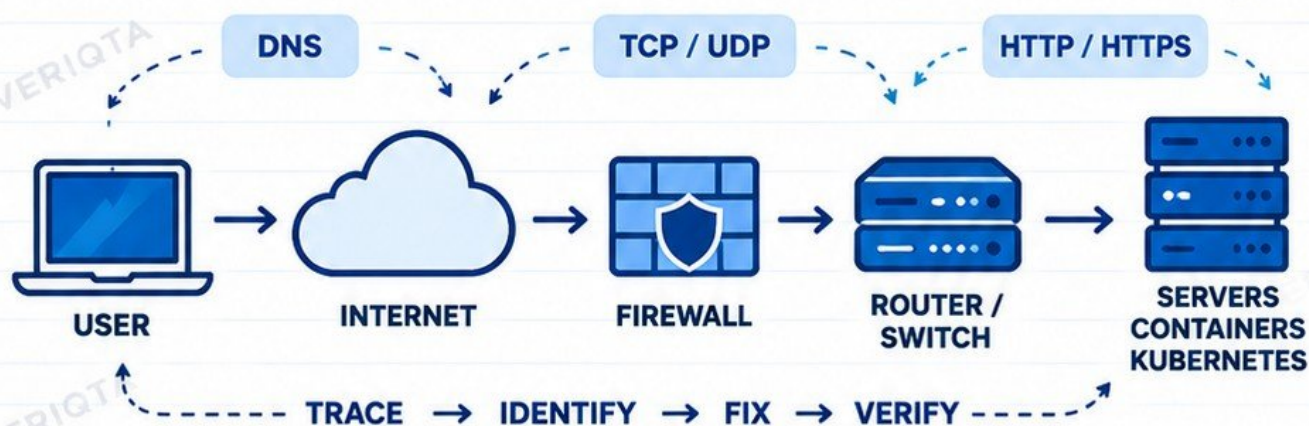


NETWORKING TROUBLESHOOTING FOR **DEVOPS** HANDBOOK

From Connection Failures to
Production Network Diagnosis




REAL
TROUBLESHOOTING
SCENARIOS


PRACTICAL
COMMANDS


PRODUCTION
BEST PRACTICES


CLOUD, CONTAINER
AND KUBERNETES
NETWORKING


BUILD REAL
DEVOPS SKILLS

**OBSERVE. DIAGNOSE. SOLVE.
KEEP SYSTEMS RUNNING.**

Better
Engineers
Stronger
Systems

HOW DEVOPS ENGINEERS TROUBLESHOOT NETWORKS

From Connection Failures to Production Network Diagnosis

1 WHY NETWORK PROBLEMS FEEL DIFFICULT

- Many components are involved (client, network, server, firewall, DNS, etc).
- A failure can happen at different layers.
- The error message is not always clear.
- The real cause is often different from what it looks like.
- Small changes can have a big impact.

2 SYMPTOMS vs ROOT CAUSES

COMMON SYMPTOMS	POSSIBLE ROOT CAUSES
Cannot reach a website →	DNS issue, network route, firewall, server down
Connection refused →	Service not running, wrong port, firewall block
Connection timed out →	Network path blocked, firewall, routing issue
Works by IP but not by name →	DNS problem
Intermittent connectivity →	Network instability, load balancer, firewall, routing

3 CLIENT, NETWORK, SERVER THINKING

Always think in three parts:



CLIENT

- Is the client working?
- Is DNS working?
- Can it reach the network?



NETWORK

- Is the traffic allowed?
- Is the route correct?
- Are firewalls blocking?
- Is there network instability?



SERVER

- Is the server up?
- Is the service running?
- Is the correct port open?
- Is the application healthy?

4 START WITH EVIDENCE, NOT ASSUMPTIONS



- Check the actual error message.
- Test from the client first.
- Use the right diagnostic commands (ping, curl, dig, etc).
- Identify where it fails.
- Collect evidence before making changes.
- Avoid guessing or making random fixes.

5 THE TROUBLESHOOTING LOOP

Follow a clear and repeatable process:



WHAT CHANGED?

Recent deployments, config changes, network updates?



WHAT FAILS?

Identify the exact symptom and error message.



WHERE DOES IT FAIL?

Client, network, server, or application layer?



WHY?

Use evidence and diagnostic commands to find the root cause.



FIX AND VERIFY

Apply the right fix and confirm it works.

6 JUNIOR ENGINEER TIP



Do not start randomly changing firewall rules.

Understand the problem, collect evidence, and identify the real cause first. Blindly opening ports can create security risks and still not fix the issue.

“TROUBLESHOOT WITH LOGIC, NOT LUCK.”

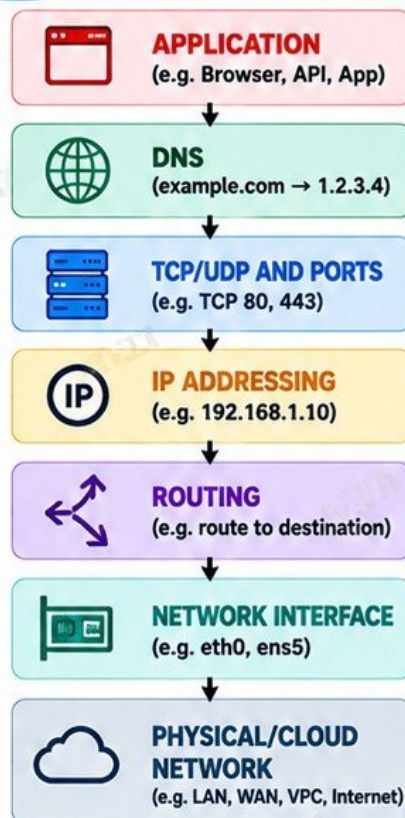
THE NETWORK TROUBLESHOOTING MENTAL MODEL

Understand the layers. Find the failure. Fix it faster.

1 THE NETWORK LAYERS

- 7 APPLICATION LAYER**
The application you use (e.g. web app, API, database).
- 6 DNS**
Converts domain names to IP addresses.
- 5 TCP/UDP AND PORTS**
Controls communication between applications.
- 4 IP ADDRESSING**
Identifies devices on the network (e.g. IPv4, IPv6).
- 3 ROUTING**
Decides the path packets take to the destination.
- 2 NETWORK INTERFACE**
Sends and receives packets (e.g. eth0, ens5).
- 1 PHYSICAL/CLOUD NETWORK**
The actual network infrastructure (cables, switches, routers, or cloud network like VPC).

2 LAYER DIAGRAM (TOP DOWN)



3 TOP-DOWN vs BOTTOM-UP

TOP-DOWN APPROACH



- ✓ Start from the application (user's perspective).
- ✓ Check each layer in order.
- ✓ Useful when the symptom is end-to-end (e.g. website not reachable).
- ✓ Helps you quickly find where it breaks.

BOTTOM-UP APPROACH



- ✓ Start from the network infrastructure.
- ✓ Work your way up the stack (layer by layer).
- ✓ Useful when you are on the server or inside the network.
- ✓ Helps you isolate infrastructure and configuration issues.

4 HOW TO IDENTIFY WHICH LAYER IS FAILING



- 1 Understand the symptom.
- 2 Test each layer with the right tools.
- 3 Compare expected vs actual results.
- 4 Use error messages and logs as clues.
- 5 Isolate the failing layer before making changes.

5 COMMON QUESTIONS AT EACH LAYER

Application	Is the application working? Any error codes?
DNS	Does the domain resolve to the correct IP?
TCP/UDP & Ports	Is the port open and accepting connections?
IP Addressing	Is the IP address correct and reachable?
Routing	Is the traffic going to the right destination?
Network Interface	Is the interface up and configured correctly?
Physical/Cloud Network	Is the underlying network working (LAN, VPC, Internet)?

6 JUNIOR ENGINEER TIP



Do not start randomly changing firewall rules.

- ✓ Find the real cause first.
- ✓ Use evidence, not assumptions.
- ✓ Test and validate each layer.
- ✓ Make small, targeted changes.
- ✓ Document what you did.

YOUR ESSENTIAL NETWORK TROUBLESHOOTING TOOLKIT

Right tools. Real answers. Faster troubleshooting.

*Diagnose
Investigate
Resolve
Repeat*

1 ESSENTIAL NETWORK TROUBLESHOOTING COMMANDS

#	COMMAND	WHAT IT DOES	WHAT IT PROVES	EXAMPLE
1	<code>ip addr</code>	Shows IP addresses on interfaces	Which IP addresses are assigned, interface state	<code>ip addr</code>
2	<code>ip link</code>	Shows network interface details	Interface is up or down, link status	<code>ip link</code>
3	<code>ip route</code>	Shows routing table	Where traffic is sent (routes, gateways)	<code>ip route</code>
4	<code>ping</code>	Tests basic network connectivity	Reachability to a host (ICMP)	<code>ping 8.8.8.8</code>
5	<code>ss</code>	Shows listening and active sockets	Which ports are open and what is connected	<code>ss -lntp</code>
6	<code>curl</code>	Tests HTTP/HTTPS endpoints	Application is reachable, HTTP response, TLS details	<code>curl -v https://example.com</code>
7	<code>dig</code>	Queries DNS servers	DNS records, resolution, and DNS issues	<code>dig example.com</code>
8	<code>nslookup</code>	Simple DNS lookup tool	Domain name resolution and DNS server	<code>nslookup example.com</code>
9	<code>tracert</code>	Shows the network path (hops)	Where traffic goes and where it stops	<code>tracert example.com</code>
10	<code>tracert</code>	Traces path (alternative to tracert)	Path to destination (works even if tracert is blocked)	<code>tracert example.com</code>
11	<code>nc</code>	Tests port connectivity	Whether a specific port is open	<code>nc -vz example.com 80</code>
12	<code>tcpdump</code>	Captures and analyzes packets	Actual network traffic (requests, responses, errors)	<code>tcpdump -i eth0 port 80</code>

2 CHOOSING THE RIGHT COMMAND

TASK / PROBLEM	USE THIS COMMAND
Check IP address	<code>ip addr</code>
Check interface status	<code>ip link</code>
Check routing / gateway	<code>ip route</code>
Test if a host is reachable	<code>ping</code>
Check if a port is open	<code>nc, ss</code>
Test a web application	<code>curl</code>
Check DNS resolution	<code>dig, nslookup</code>
Find the network path	<code>tracert, tracert</code>
Capture and analyze traffic	<code>tcpdump</code>

4 JUNIOR ENGINEER TIPS

- ✓ Start with simple commands (ping, curl, nslookup).
- ✓ Move layer by layer (don't jump to firewalls).
- ✓ Use the right tool for the problem.
- ✓ Always check both client and server side.
- ✓ Read and understand the output (it gives clues).
- ✓ Document what you did and what you found.
- ✓ Practice these commands regularly.

**THE RIGHT
COMMAND
AT THE RIGHT TIME
SAVES HOURS.**

3 TROUBLESHOOTING WORKFLOW

- 1 IDENTIFY THE SYMPTOM**
What is not working?
- 2 USE THE RIGHT TOOL**
Test each layer (DNS, network, port, etc.).
- 3 ANALYZE THE RESULTS**
What does the output tell you?
- 4 FIND THE ROOT CAUSE**
Which layer is failing?
- 5 APPLY THE FIX**
Make the necessary change.
- 6 VERIFY**
Test again to confirm it works.

*Observe
Test
Isolate
Fix
Verify*

5 KEY TAKEAWAY



Each command helps you answer a specific question. Use them together, follow a structured approach, and you will find and fix network issues faster.

"THE SERVER IS DOWN" START HERE

Same
Problems
Stronger
Engineers

Don't guess. Investigate. Find the real cause.

1 CONFIRM THE ACTUAL SYMPTOM

- What exactly is not working?
- Is it a website, API, database, or server access (SSH)?
- When did it start?
- Is it affecting everyone or just you?
- Has anything changed recently?
- Check error messages, logs and user reports.

2 IS THE HOST REACHABLE?

```
ping <IP or hostname>
ping 8.8.8.8
```

- Tests basic network connectivity (ICMP).
- Tells you if the host is reachable.
- If ping fails, the host may be down, blocked, or unreachable.

3 IS DNS CORRECT?

```
nslookup <domain>
nslookup example.com
```

```
dig <domain>
dig example.com
```

- Checks if the domain resolves to the correct IP.
- If DNS is wrong, the application will not be reachable even if the server is up.

4 IS THE APPLICATION RUNNING?

```
systemctl status <service>
systemctl status nginx
```

- Checks if the application service is running.
- Look for "active (running)".
- If the service is stopped or failed, start it or check the logs.

5 IS THE PORT LISTENING?

```
ss -lntp
ss -lntp | grep :80
```

- Shows listening ports and the process using them.
- Confirms if the application is listening on the expected port (e.g. 80, 443, 3306).
- If the port is not listening, the application is not reachable.

6 IS TRAFFIC REACHING THE HOST?

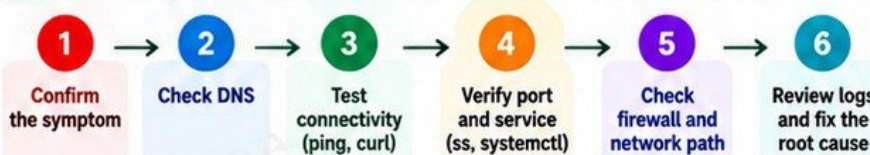
```
curl -v http://<IP or domain>
curl -v http://example.com
```

- Tests actual connectivity to the application.
- Shows detailed connection steps (DNS, TCP, TLS, HTTP).
- Helps identify if the request reaches the server and what response is returned.

7 WHY "SERVER DOWN" CAN MEAN DIFFERENT THINGS

SCENARIO	WHAT IT MEANS	EXAMPLE SYMPTOM	WHAT TO CHECK
Host is unreachable	The server is not responding on the network	ping fails	Network, routing, firewall
DNS issue	Domain does not resolve or resolves to wrong IP	nslookup/ dig fails or wrong IP	DNS records, resolver config
Application not running	Service is stopped or crashed	Connection refused	systemctl status, logs
Port not listening	Application is not bound to the port	Cannot connect to port	ss -lntp, application config
Firewall blocking	Traffic is blocked	Timeout	Firewall rules, security groups
Application error	Server is up but app returns error	HTTP 500, 503	Application logs
Resource issue	Server is overloaded (CPU, memory)	Slow or no response	top, free, df, logs

8 TROUBLESHOOTING FLOW



9 JUNIOR ENGINEER TIP



Do not start randomly changing firewall rules.

- Investigate and understand the real cause.
- Use evidence, not assumptions.
- Try the simplest checks first.
- Changing firewall rules blindly can create security risks and still not fix the issue.

Learn
Practice
Grow



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

ENGINEERS TODAY. A STRONGER TOMORROW.

REAL
SKILLS
REAL
OPPORTUNITIES

*Same
Problems
Stronger
Engineers*

IP ADDRESS AND INTERFACE TROUBLESHOOTING

Understand your network identity. Fix connectivity issues faster.

1 UNDERSTANDING ASSIGNED IP ADDRESSES

- An IP address identifies a device on a network.
- It can be assigned statically or dynamically (e.g. via DHCP).
- Each interface can have one or more IP addresses (IPv4 and/or IPv6).
- An IP address has two parts: network portion and host portion.
- Incorrect IP configuration can prevent network connectivity.

2 LOOPBACK vs PRIVATE vs PUBLIC IP

TYPE	RANGE (IPv4)	USE CASE	EXAMPLE
Loopback	127.0.0.0/8	Used to test the local machine (never leaves the host)	127.0.0.1
Private	10.0.0.0/8 172.16.0.0/12 192.168.0.0/16	Used in internal networks (not routable on the internet)	192.168.1.10
Public	All other IPs	Routable IPs the internet (assigned by ISP or cloud provider)	8.8.8.8

3 INTERFACE UP/DOWN STATE

- A network interface must be UP to send and receive traffic.
- An interface can be:

UP

- Interface is active.
- Can send and receive packets.

DOWN

- Interface is inactive.
- Cannot send or receive packets.
- May be administratively down or due to a problem.

4 USEFUL COMMANDS

\$ ip addr

- Shows IP addresses assigned to all interfaces.
- Displays interface state, IP (IPv4/IPv6), subnet mask, etc.

\$ ip link

- Shows interface details and status (UP or DOWN).
- Useful for checking if an interface is enabled and working.

5 EXAMPLE OUTPUT

\$ ip addr

```
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 192.168.1.10/24 brd 192.168.1.255
    inet6 fe80::216:3eff:fe12:3456/64
```

\$ ip link

```
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP>
    state UP mode DEFAULT group default
3: ens6: <BROADCAST,MULTICAST>
    state DOWN mode DEFAULT group default
```

6 COMMON ISSUES AND FIXES

ISSUE	SYMPTOM	POSSIBLE FIX
Missing IP address	No IP assigned to interface	Check DHCP, network config
Interface down	Cannot send/receive traffic	Bring interface up (ip link set <iface> up)
Wrong subnet	Cannot reach other hosts	Assign correct IP and subnet mask
Multiple interfaces	Traffic going out the wrong interface	Check routing table (ip route)
Duplicate IP	Intermittent connectivity	Find and remove the duplicate IP
Cloud VM issues	No internet or internal access	Check cloud network settings, security groups, route tables

7 MULTIPLE INTERFACES

- Servers and cloud VMs often have multiple network interfaces (e.g. eth0, eth1, ens5).
- Each interface can be on a different network or subnet.
- Make sure the correct interface is used for the intended traffic.
- Check the routing table to see which interface is used for each destination.

8 DUPLICATE IP SYMPTOMS

- Intermittent connectivity (sometimes works, sometimes not).
- ARP conflicts in logs.
- Devices become unreachable randomly.
- Can cause packet loss and strange network behaviour.
- Use tools like `arp -a` or check switch/router logs to identify conflicts.

9 CLOUD VM INTERFACE CONSIDERATIONS

- Cloud VMs (AWS, Azure, GCP, etc.) may have multiple network interfaces (primary and secondary).
- Check that the correct interface has the expected IP (private or public).
- Ensure security groups, route tables, and network ACLs allow the traffic.
- Use the cloud console or CLI to verify network interface settings.
- Some cloud providers use DHCP, so IPs can change if the instance is stopped/started.



JUNIOR ENGINEER TIP

Always verify your IP configuration and interface status before troubleshooting higher layers (DNS, firewall, or application).

*Small
Checks
Big
Results*


@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

ROUTING FAILURES

WHEN PACKETS CANNOT FIND THE PATH

Understand the route. Fix the path. Restore connectivity.

*Different
Paths
Same
Destination*

1 WHAT ROUTING ACTUALLY DECIDES

- Routing decides the best path for packets to reach the destination IP.
- It uses routing tables to match the destination IP with a route.
- Each route specifies the next hop (gateway) and the outgoing interface.
- If no matching route exists, packets cannot be delivered.
- Routing does not open ports or allow traffic — it only decides the path.

2 ROUTING TABLES

- A routing table contains multiple routes to different networks.
- Each route has a destination, a gateway (next hop) and an interface.
- The system chooses the most specific matching route (longest prefix match).

Destination	Gateway	Interface	Purpose
0.0.0.0/0	192.168.1.1	eth0	Default route (all traffic)
192.168.1.0/24	-	eth0	Local network
10.0.0.0/16	192.168.1.1	eth0	Remote network

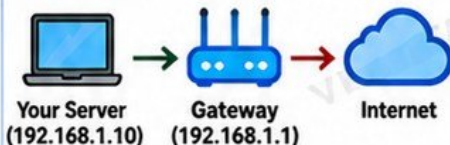
3 DEFAULT ROUTES

- The default route (0.0.0.0/0) is used when no other route matches.
- It usually points to a gateway (e.g. your router or cloud NAT gateway).
- Without a default route, the server cannot reach networks outside its local network (e.g. the internet).

```
$ ip route | grep default
default via 192.168.1.1 dev eth0
```

4 GATEWAYS

- A gateway is the next hop router that forwards packets to another network.
- On a local network, the gateway is often your router or cloud virtual router.
- In the cloud, the gateway can be an Internet Gateway, NAT Gateway or a virtual router.



5 USEFUL COMMANDS

```
$ ip route
```

- Shows the current routing table.

```
$ ip route get 8.8.8.8
```

- Shows the exact route the system will use to reach a specific IP.
- Helps you verify the next hop and outgoing interface.

EXAMPLE OUTPUT

```
$ ip route get 8.8.8.8
8.8.8.8 via 192.168.1.1 dev eth0
src 192.168.1.10
cache
```

6 COMMON ROUTING ISSUES



Missing default route

- No route to reach external networks.
- Symptoms: cannot reach the internet.
- Fix: add the correct default route.



Incorrect gateway

- Gateway IP is wrong or unreachable.
- Symptoms: destination not reachable.
- Fix: check and set the correct gateway.



Wrong subnet route

- Traffic is sent to the wrong network or interface.
- Symptoms: can't reach specific network.
- Fix: correct the route or subnet mask.



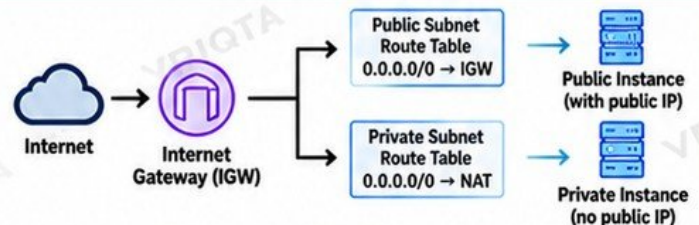
Asymmetric routing

- Request and response take different paths.
- Symptoms: intermittent connectivity, timeouts.
- Fix: ensure consistent routing (use the same gateway/path).

7 HOW CLOUD ROUTE TABLES ENTER THE PICTURE

- In cloud environments (AWS, Azure, GCP), route tables determine where traffic from subnets is sent.
- Public subnets often route to an Internet Gateway (IGW).
- Private subnets often route to a NAT Gateway for internet access.
- Route tables control traffic between subnets, VPCs and on-premises networks (via VPN or Direct Connect).
- Security groups and network ACLs allow or block traffic, but routing decides the path.
- Always check the correct route table for the subnet your instance or workload is in.

EXAMPLE: AWS VPC ROUTING



8 JUNIOR ENGINEER TIP



If a server cannot reach a destination, always check the routing table first.

A missing or incorrect route is a common cause of network failures and is often faster to fix than firewall or application issues.

**“ NO ROUTE.
NO CONNECTION. ”**

DNS TROUBLESHOOTING

"IT WORKS BY IP BUT NOT BY NAME"

Find the DNS issue. Restore name resolution. Get back online.

*Names
Matter
Fix DNS
Faster*

1 DNS RESOLUTION FAILURES

- DNS translates domain names to IP addresses.
- If DNS fails, applications cannot find the correct server.
- Symptoms: website not reachable by name, but works by IP.
- Causes: misconfiguration, wrong records, DNS server issues, network problems, or caching.



2 /etc/resolv.conf

```
$ cat /etc/resolv.conf
nameserver 8.8.8.8
nameserver 1.1.1.1
search example.com
options edns0 trust-ad
```

- Specifies which DNS servers to use.
- Can be configured manually or via DHCP.
- May be overwritten by system services (e.g. systemd-resolved, NetworkManager).

3 RESOLVER CONFIGURATION

- Your system (or container) uses a resolver to query DNS servers.
- Configuration sources:
 - /etc/resolv.conf
 - systemd-resolved
 - NetworkManager
 - DHCP (cloud or on-premises)
 - Custom DNS servers (e.g. in VPC/VNet)
- Check active DNS settings:



```
$ resolvectl status
```

4 ESSENTIAL DNS COMMANDS

```
$ dig example.com
```

- Queries DNS and shows detailed results.
- Best for troubleshooting.

```
$ nslookup example.com
```

- Simple DNS lookup tool.
- Shows the resolved IP and DNS server used.

```
$ getent hosts example.com
```

- Uses the system resolver (same as applications).
- Useful to test name resolution exactly as your system sees it.

5 COMMON DNS RECORD TYPES

TYPE	PURPOSE
A	Maps a domain to an IPv4 address (e.g. example.com → 1.2.3.4)
AAAA	Maps a domain to an IPv6 address (e.g. example.com → 2001:db8::1)
CNAME	Points a domain to another domain (alias)
MX	Mail server records
NS	Authoritative name servers
TXT	Text records (e.g. SPF, verification)

6 COMMON DNS ERRORS

ERROR	MEANING
NXDOMAIN	Domain does not exist.
SERVFAIL	DNS server failed to process the request.
TIMEOUT	No response from DNS server (network or server issue).
REFUSED	DNS server refused the query (permissions or policy).
NOERROR (empty result)	Valid response but no records returned.

7 WRONG DNS RECORDS

- Domain points to the wrong IP address (e.g. old server).
- Missing A or AAAA record.
- Incorrect CNAME (points to wrong name).
- TTL too high (changes take time to reflect).
- Check records with:

```
$ dig example.com
$ dig example.com A
```

8 DNS CACHING

- DNS results are cached to improve performance.
- Caching can cause old or incorrect records to be used.
- Check TTL (time to live) in dig output.
- Clear cache when needed:
 - Linux (systemd): `sudo resolvectl flush-caches`
 - macOS: `sudo dscacheutil -flushcache`
 - Windows: `ipconfig /flushdns`



9 THE KEY TEST

Compare hostname vs direct IP:

```
# Test by name
$ curl -v https://example.com
# Test by IP
$ curl -v https://1.2.3.4
```

- If it works by IP but not by name, the issue is DNS (not the server or application).
- If both fail, the problem is likely network, firewall, or the application itself.



10 DNS TROUBLESHOOTING FLOW



11 JUNIOR ENGINEER TIP



Always test name resolution before changing any application or firewall settings.

Learn
Practice
Grow

PORTS AND “CONNECTION REFUSED”

 Small
Checks
Big
Results

Understand ports. Find the service. Fix the connection.

1 WHAT A PORT REPRESENTS

- A port is a logical endpoint on a server used by applications to send and receive network traffic.
- Each port number (0–65535) represents a specific service (e.g. 22 = SSH, 80 = HTTP, 443 = HTTPS).
- Ports work with IP addresses and protocols (TCP or UDP).
- Multiple services can run on the same server using different ports.

2 LISTENING SOCKETS

- A listening socket means a service is actively waiting for incoming connections on a port.
- If a service is not listening, connection attempts will fail.
- Use `ss` to check listening sockets (TCP and UDP).



3 USEFUL COMMANDS

```
$ ss -lntp # TCP listening ports
$ ss -lunp # UDP listening ports
$ curl http://localhost:80
$ nc -vz 192.168.1.10 80
```

- `ss -lntp` shows listening TCP ports and the process using them.
- `ss -lunp` shows listening UDP ports.
- `curl` tests if a service responds.
- `nc -vz` tests if a port is open.
- Use these commands from your local machine or a remote server.

4 ss -lntp (TCP LISTENING PORTS)

```
$ ss -lntp
```

State	Recv-Q	Send-Q	Local Address:Port	Process
LISTEN	0	128	0.0.0.0:22	users:([sshd],pid=123)
LISTEN	0	128	127.0.0.1:3306	users:([mysqld],pid=456)
LISTEN	0	128	0.0.0.0:80	users:([nginx],pid=789)

- Shows all listening TCP ports.
- Reveals which process is using each port.
- 127.0.0.1 means the service is only accessible locally.
- 0.0.0.0 means the service is listening on all interfaces (local and external).

5 ss -lunp (UDP LISTENING PORTS)

```
$ ss -lunp
```

State	Recv-Q	Send-Q	Local Address:Port
UNCONN	0	0	0.0.0.0:53
UNCONN	0	0	127.0.0.1:123

- Shows all listening UDP ports.
- Used for services like DNS (53), NTP (123), DHCP (67/68) etc.

6 curl AND nc -vz

```
$ curl -v http://192.168.1.10:80
```

- Tests if the service is reachable.
- Shows detailed connection steps and HTTP response.

```
$ nc -vz 192.168.1.10 80
```

- Tests if a port is open.
- `-v` shows detailed output.
- `-z` means just scan (don't send data).

7 CONNECTION REFUSED vs TIMEOUT

CONNECTION REFUSED

- The host is reachable, but nothing is listening on the port.
- You get an immediate failure (response: RST).
- Usually means the service is not running or listening on that port.
- Example: `curl: (7) Failed to connect - Connection refused`

TIMEOUT

- No response is received.
- Packets are sent but there is no reply (dropped or blocked).
- Usually caused by a firewall, security group, or network issue.
- Example: `curl: (28) Operation timed out`

8 SERVICE RUNNING ON THE WRONG PORT

- A service may be running, but on a different port than expected.
- Check the application configuration file.
- Use `ss -lntp` to find the actual port.
- Update your client, load balancer, or firewall to use the correct port.

Example:

Service expected on port 80, but actually running on port 8080.
Fix: use the correct port or update the service configuration.

9 SERVICE BOUND ONLY TO 127.0.0.1

- If a service is bound to 127.0.0.1, it can only be accessed locally (on the same server).
- It will not be reachable from other machines, even if the firewall is open.
- To allow external access, configure the service to bind to 0.0.0.0 (or the server's private/public IP).

Example:

```
LISTEN 0 128 127.0.0.1:3306 (only local access)
```

Fix: change to 0.0.0.0:3306 in the application config.

10 0.0.0.0 EXPLAINED

- 0.0.0.0 means "listen on all interfaces" (IPv4).
- The service will accept connections from any IP (local and remote).
- This is required if you want the service to be accessible from other machines.
- Use firewall rules and security groups to restrict access to trusted IPs.

11 FINDING WHICH PROCESS OWNS A PORT

- Use `ss -lntp` to see the process name and PID.
- You can also use:

```
$ lsof -i :80
$ netstat -tulnp | grep :80
$ ps -p <PID> -f
```

- This helps you confirm which application is using the port and troubleshoot further.



JUNIOR ENGINEER TIP

- Always verify what is listening, on which IP and port, before checking firewalls.
- A simple port check can save hours of troubleshooting.

“CHECK THE PORT.
CHECK THE SERVICE.
SOLVE THE PROBLEM.”



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Small
Steps
Big
Skills

TCP TROUBLESHOOTING FROM SYN TO CONNECTION

Learn
Practice
Grow

Understand the handshake. Find the failure. Fix it faster.

1 WHAT IS TCP?

- TCP (Transmission Control Protocol) is a reliable, connection-oriented protocol.
- It ensures data is delivered in order, without loss, using a handshake, acknowledgements, and retransmissions.
- Before any data is exchanged, a TCP connection must be established using a three-way handshake.



No handshake = No communication.

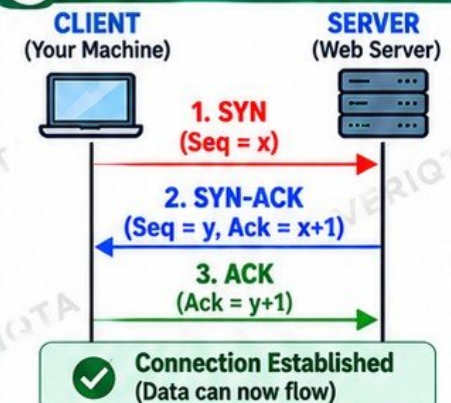
2 THE TCP THREE-WAY HANDSHAKE

- **SYN** – Client sends a request to start a connection.
- **SYN-ACK** – Server acknowledges the request.
- **ACK** – Client confirms the connection.
- Once the handshake is complete, the connection is established and data can flow in both directions.



This handshake happens very quickly (usually in milliseconds).

3 HANDSHAKE DIAGRAM



4 KEY PACKETS EXPLAINED

SYN

- Sent by client to start a connection.
- Contains initial sequence number (ISN).

SYN-ACK

- Sent by server in response.
- Acknowledges the client's SYN and has its own ISN.

ACK

- Sent by client to confirm.
- Completes the handshake.

RST

- Sent when a connection is immediately rejected or closed.
- Usually means the port is closed or the service is not available.

5 RETRANSMISSIONS

- If a packet (SYN, SYN-ACK, or ACK) is lost, TCP will retransmit it.
- Multiple retransmissions increase the connection time.
- If no response is received after several attempts, the connection fails with a timeout.



High retransmissions often indicate network issues such as packet loss, firewall rules, or an unreachable host.

6 COMMON CONNECTION FAILURES

Connection Refused

- You get an immediate RST from the server.
- Usually means the port is closed or no service is listening.

Timeout

- No response (SYN or SYN-ACK).
- Usually caused by firewall rules, network issues, or the server is unreachable.

Connection Reset

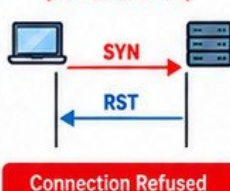
- Connection was established but later reset (RST).
- Can be caused by the server, firewall, or application closing the connection.

7 PACKET FLOW EXAMPLES

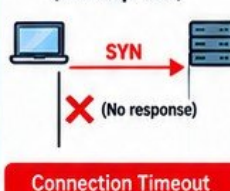
Successful Connection



Connection Refused (Immediate RST)



Connection Timeout (No Response)



8 WHY THE HANDSHAKE MATTERS

- Helps you quickly identify where the failure is occurring (client, network, or server).
- Shows whether the server is reachable and listening.
- Distinguishes between a network issue, firewall block, or application problem.
- Understanding the handshake saves time and prevents guesswork.



If you understand the handshake, you can troubleshoot most connection issues logically and faster.

9 JUNIOR ENGINEER TIP



Before changing firewall rules or application configs, always check the handshake. Use tools like `nc`, `curl`, and `tcpdump` to see what is actually happening.

“UNDERSTAND THE FLOW.
FIND THE BREAK.
FIX THE PROBLEM.”



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
PRACTICE
BUILD
GROW

FIREWALL AND SECURITY RULE TROUBLESHOOTING

Learn
Practice
Grow

Secure
Deploy
Scale

Find the block. Fix the rule. Keep it secure.

1 HOST FIREWALL vs CLOUD FIREWALL

HOST FIREWALL



- Runs on the server/VM.
- Controls traffic in and out of the local machine.
- Examples: nftables, iptables, ufw.
- Useful for additional protection.

CLOUD FIREWALL



- Runs in the cloud provider network.
- Controls traffic to and from cloud resources (e.g. EC2, RDS, ALB).
- Examples: AWS Security Groups, Network ACLs.
- Acts before traffic reaches your server.

2 LINUX FIREWALL TOOLS

nftables

- Modern Linux firewall tool.
- Replaces iptables.
- More flexible and powerful.

iptables

- Older firewall tool.
- Still widely used.
- Know basic commands for troubleshooting legacy systems.

ufw

- User-friendly firewall for Ubuntu.
- Simplifies iptables/nftables rules.
- Common commands:
`sudo ufw status`
`sudo ufw allow 22/tcp`

3 AWS SECURITY GROUPS



- Stateful firewall for AWS resources (e.g. EC2, RDS, ALB).
- Controls inbound and outbound traffic.
- Rules allow specific IP ranges (CIDR), ports and protocols.
- If a rule is missing, traffic will be blocked.
- Use least privilege (only allow what is needed).

4 NETWORK ACL CONCEPTS (AWS)



- Stateless firewall at the subnet level.
- Both inbound and outbound rules must be explicitly allowed.
- Evaluated in order (rule number).
- Used for an additional layer of security.
- Can block traffic even if Security Groups allow it.

5 INBOUND vs OUTBOUND TRAFFIC



INBOUND

- Traffic coming into your server or resource.
- Example: Users accessing your web app (port 80/443).



OUTBOUND

- Traffic leaving your server or resource.
- Example: Your server connecting to an external API (port 443).

6 STATEFUL vs STATELESS FILTERING



STATEFUL (e.g. Security Groups)

- Remembers the state of connections.
- If you allow inbound, the response is automatically allowed.
- Easier to manage.



STATELESS (e.g. Network ACLs)

- Does not remember connection state.
- You must allow both inbound and outbound traffic.
- More configuration required.

7 COMMON FIREWALL ISSUES

WRONG SOURCE CIDR



- Traffic is blocked because the source IP or CIDR is not allowed.
- Example: Allowing 192.168.1.0/24 but traffic comes from 10.0.0.0/24.

MISSING PORT RULE



- The required port is not allowed in the firewall.
- Example: Port 80 not open for HTTP, or port 443 not open for HTTPS.
- Always check both inbound and outbound rules.

SERVICE BOUND ONLY TO 127.0.0.1



- The service is only listening on localhost.
- External traffic is blocked even if firewall rules are correct.
- Fix: Configure the service to listen on 0.0.0.0.

8 WHY 0.0.0.0/0 IS NOT TROUBLESHOOTING



- Allowing everything (0.0.0.0/0) might make it work, but it is not a fix.
- It creates serious security risks.
- It bypasses the real problem.
- Always identify the correct source IP or CIDR and allow only what is needed (least privilege).



Opening all ports to 0.0.0.0/0 is not troubleshooting. It is a security risk.

9 EXAMPLE: ALLOW SSH FROM A SPECIFIC IP

```
# Using ufw
sudo ufw allow from 203.0.113.10 to any port 22

# Using nftables
sudo nft add rule inet filter input ip saddr 203.0.113.10 tcp dport 22 accept
```

10 TROUBLESHOOTING CHECKLIST

- ✓ Verify the service is running and listening.
- ✓ Check host firewall rules (nftables/iptables/ufw).
- ✓ Check cloud firewall rules (Security Groups).
- ✓ Check Network ACLs (if in AWS).
- ✓ Confirm the source IP/CIDR is allowed.
- ✓ Verify the correct port and protocol.
- ✓ Test from the client (e.g. nc, curl, ssh).
- ✓ Avoid opening everything to 0.0.0.0/0.

“ FIND THE BLOCK. ALLOW ONLY WHAT YOU NEED. STAY SECURE. ”

Learn
Practice
Grow

HTTP AND HTTPS

TROUBLESHOOTING WITH CURL

Real
Scenarios
Stronger
Engineers

Test. Analyse. Identify. Fix.

1 USING curl -v (VERBOSE MODE)

```
$ curl -v https://example.com
```

- Shows the full request and response details.
- Displays DNS resolution, IP address, connection steps, request headers, response headers and body (if any).
- Very useful for troubleshooting.

What to look for:

- DNS resolution
- Connection establishment
- TLS handshake (for HTTPS)
- HTTP request and response
- Error messages



2 USING curl -I (HEADERS ONLY)

```
$ curl -I https://example.com
```

- Sends a HEAD request.
- Shows only the response headers (faster and cleaner).
- Useful for checking status codes, redirects, and headers.

Example output:

```
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 02 Sep 2025 10:00:00 GMT
Content-Type: text/html
Content-Length: 1256
```

3 HTTP RESPONSE CODES

Code Range	Meaning
1xx	Informational (e.g. 100 Continue)
2xx	Success (e.g. 200 OK)
3xx	Redirection (e.g. 301, 302)
4xx	Client Error (e.g. 400, 404, 403)
5xx	Server Error (e.g. 500, 502, 503)

4 CONNECTION FAILURES vs APPLICATION ERRORS



CONNECTION FAILURES (Network/Infrastructure)

- Cannot resolve host (DNS issue)
- Connection refused (nothing listening on the port)
- Connection timed out (blocked by firewall or network issue)
- No route to host
- TLS handshake fails

These happen before the HTTP request is completed.



APPLICATION ERRORS (HTTP Response Received)

- You get an HTTP response code (e.g. 404, 500).
- This means the network connection worked, but the application returned an error.
- Check the response body for details.

If you see an HTTP status code, the network is working.

5 HOST HEADERS

- Some applications respond differently based on the Host header.
- Use -H to set a custom host header.

```
$ curl -v -H "Host: example.com" \
http://192.168.1.10
```

Why use this?

- Testing virtual hosts
- Troubleshooting load balancers or reverse proxies
- Accessing a service by IP but using the correct domain name

6 REDIRECTS

- HTTP 3xx codes mean the request is being redirected.
- Use -L to follow redirects automatically.

```
$ curl -v -L
http://example.com
```

Common redirects:

- 301 – Moved permanently
- 302 – Found (temporary)
- 307 – Temporary redirect (keeps method)
- 308 – Permanent redirect (keeps method)

7 TLS HANDSHAKE CLUES (HTTPS)



In verbose mode (-v), curl shows the TLS handshake:

- TLS version (e.g. TLSv1.2, TLSv1.3)
- Cipher suite
- Server certificate details
- Whether the certificate is valid

Example output (simplified):

```
* TLSv1.3 (IN), Server hello
* SSL certificate verify ok
* Server certificate:
subject: CN=example.com
start date: Sep 1 00:00:00 2024 GMT
expire date: Sep 1 00:00:00 2025 GMT
```

8 CERTIFICATE FAILURES



Common certificate errors:

- certificate expired
- self signed certificate
- unable to get local issuer certificate
- hostname mismatch
- certificate revoked

```
$ curl -v https://example.com
```

Important:

- Do not use -k in production.
- k skips certificate verification and hides the real problem.
- Fix the certificate issue instead.

9 TESTING AN ENDPOINT DIRECTLY



- Test the service by IP and by domain name.
- Compare the results.

```
$ curl -v http://192.168.1.10:80
$ curl -v http://example.com
```

If IP works but domain fails:

- Likely a DNS or host header issue.
- If neither works: check network, firewall, or service.

10 SEPARATING NETWORK PROBLEMS FROM HTTP/APPLICATION PROBLEMS



- Check DNS resolution (dig, nslookup, or curl output).
- Check if the port is reachable (curl, nc).
- Look for TLS errors (if HTTPS).
- If you get an HTTP status code, read the response and check application logs.
- Use the right tool for the right layer (network vs application).

“DIAGNOSE LAYER BY LAYER.
CURL SHOWS THE TRUTH.
FIND THE REAL PROBLEM,
NOT JUST A WORKAROUND.”



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

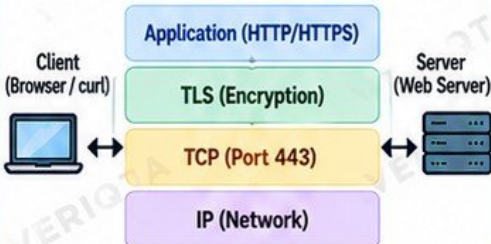
LEARN
PRACTICE
BUILD
GROW

TLS AND CERTIFICATE FAILURES

Understand the trust. Find the issue. Fix it securely.

1 WHERE TLS SITS IN THE CONNECTION

TLS sits between TCP and the application (e.g. HTTPS).

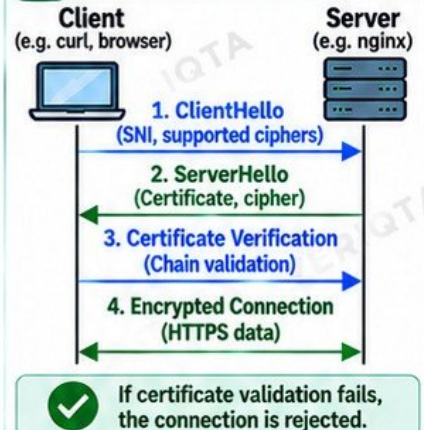


TLS secures the data after a TCP connection is established.

2 COMMON CERTIFICATE FAILURES

-  **Certificate expiration**
The certificate is no longer valid.
-  **Hostname mismatch**
The domain name does not match the certificate.
-  **Certificate chains**
Intermediate certificates are missing or not trusted.
-  **Unknown certificate authority**
The certificate is not issued by a trusted CA.
-  **System clock problems**
If your server's clock is incorrect, certificates may appear invalid.

3 TLS HANDSHAKE FLOW



4 USING curl -v

```
$ curl -v https://example.com
* Trying 93.184.216.34:443...
* Connected to example.com (93.184.216.34) port 443
* ALPN: curl offers h2,http/1.1
* TLS handshake, Client hello (1):
* TLS handshake, Server hello (2):
* SSL certificate verify ok.
> GET / HTTP/1.1
> Host: example.com
```

- Shows the full TLS handshake process.
- Displays certificate details.
- Helps identify where the failure occurs.

5 USING openssl s_client

```
$ openssl s_client -connect example.com:443
-servername example.com

CONNECTED(00000000000000000000000000000000)
depth=2 0 = GlobalSign
depth=1 0 = R3, CN = Let's Encrypt
depth=0 CN = example.com
verify return:1
---
SSL-Session:
    Protocol  : TLSv1.3
    Cipher    : TLS_AES_256_GCM_SHA384
```

- Shows detailed certificate information.
- Helps check the certificate chain.
- Use `-servername` for SNI (Server Name Indication).

6 SNI (SERVER NAME INDICATION)

- SNI allows the client to tell the server which domain it is requesting.
- Important when a server hosts multiple domains on the same IP address (virtual hosting).
- Without SNI, the server may return the wrong certificate, causing a hostname mismatch error.

Example:

```
$ openssl s_client -connect example.com:443
-servername example.com
```

Always use `-servername` when testing with `openssl` for domain-based sites.

7 WHY curl -k IS NOT A PRODUCTION FIX

```
$ curl -k https://example.com
```

- `-k` (or `--insecure`) ignores certificate validation errors.
- It allows the connection even if the certificate is expired, self-signed, or has a hostname mismatch.
- Useful for quick testing, but it hides real problems.

 **Do not use `-k` in production.**
Fix the certificate issue instead of ignoring it.

8 SAFE CERTIFICATE TROUBLESHOOTING

-  Check the certificate expiration date.
-  Verify the domain name matches the certificate (Common Name or SAN).
-  Ensure the full certificate chain is present.
-  Confirm the issuing CA is trusted by your system.
-  Check the server's date and time.
-  Use `openssl s_client` and `curl -v` for detailed output.
-  Test from different machines (local and remote).
-  If needed, update or renew the certificate.
-  Avoid disabling validation except for temporary testing in non-production environments.

9 COMMON ERROR MESSAGES

Error Message	Meaning
certificate expired	The certificate is no longer valid.
hostname mismatch	The domain does not match the certificate.
unable to get local issuer certificate	The issuing CA is not trusted or missing.
self signed certificate	The certificate is not from a trusted CA.
certificate verify failed	General certificate validation failure.
SSL: wrong version number	The server is not speaking TLS/SSL.

10 JUNIOR ENGINEER TIP



- Always test with the real domain name (not just the IP).
- Check the full chain, not just the server certificate.
- Verify the server's time and time zone.
- Use the right tools (`curl`, `openssl`, browser) and read the error messages carefully.
- Fix the root cause. Do not disable security checks.

“ A VALID CERTIFICATE
BUILDS TRUST.
FIX THE CAUSE.
DON'T BYPASS IT. ”

TRACING THE PATH, WHERE DOES TRAFFIC STOP?

Follow the packets. Find the break. Fix the problem.

1 TRACEROUTE AND TRACEPATH

traceroute

Shows the path packets take to reach a destination using ICMP (or UDP/TCP).

```
$ traceroute example.com
```

- ✓ Shows each hop (router)
- ✓ Displays latency (RTT)
- ✓ Helps identify where packets stop

tracepath

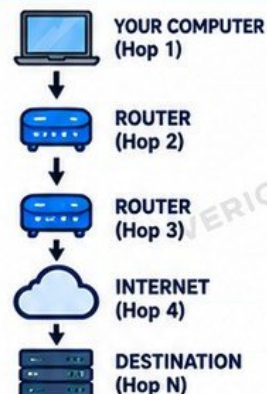
Similar to traceroute but uses UDP and does not require root privileges on most systems.

```
$ tracepath example.com
```

- ✓ Shows the path
- ✓ Displays MTU information
- ✓ Useful on restricted environments

2 UNDERSTANDING HOPS

- Each hop is a router between your machine and the destination.
- Hops show the path, not just the final IP.
- More hops do not always mean a problem.
- Each hop replies with its IP and the time taken (latency).



3 EXAMPLE OUTPUT (traceroute)

```
$ traceroute example.com
traceroute to example.com (93.184.216.34), 30 hops max
 1  192.168.1.1      1.2 ms   1.1 ms   1.3 ms
 2  10.0.0.1          5.4 ms   5.2 ms   5.6 ms
 3  203.0.113.1       12.8 ms  13.1 ms  12.6 ms
 4  198.51.100.1      28.3 ms  27.9 ms  28.1 ms
 5  93.184.216.34     32.7 ms  32.5 ms  32.8 ms
```

- ✓ Each line is a hop.
- ✓ The times show how long it took to get a response (latency).
- ✓ The last hop is the destination.

4 LATENCY BETWEEN HOPS



- Latency is the time (in milliseconds) for a packet to reach a hop and get a response.
- Small latency is normal for local networks.
- Higher latency is expected over the internet.
- Large or increasing latency can indicate congestion or a routing issue.



Typical Latency Reference (approximate)

Same network (LAN)	< 1 ms
Same region (cloud)	1 – 10 ms
Different region (cloud)	10 – 100 ms
Different country	50 – 300 ms+

5 PACKET LOSS INTERPRETATION



- Packet loss means some probes did not get a response.
- It is shown as missing responses or very high timeouts.
- Occasional packet loss can be normal.
- Consistent packet loss usually indicates a network issue.



Packet loss at an early hop often affects everything after it. Loss at a later hop may only affect specific routes.

6 ICMP FILTERING



- Many networks block or rate-limit ICMP (which traceroute uses).
- This can cause hops to show *** even when the network is working.
- Firewalls, cloud providers, and routers often filter ICMP for security.
- Try using tracepath or TCP-based traceroute (e.g. traceroute -T).
- Always test actual application connectivity (e.g. curl, ping, nc).

7 WHY *** DOES NOT MEAN A BROKEN NETWORK

- A hop may not reply due to ICMP filtering.
- The router may deprioritize responses.
- It does not mean packets are not being forwarded.
- If later hops respond, the path is working.
- Always confirm with actual application tests.

8 LOCAL NETWORK vs INTERNET PATH



LOCAL NETWORK

- Fewer hops
- Lower latency
- Usually more stable
- You control most of it (e.g. office or VPC)



INTERNET PATH

- More hops
- Higher latency
- Less predictable
- Involves multiple networks you do not control

9 CLOUD ROUTING CONSIDERATIONS



- Traffic may go through multiple cloud networks.
- NAT Gateways, Internet Gateways, and Transit Gateways add extra hops.
- Routing policies can change the path.
- Use cloud provider tools (e.g. VPC flow logs, route tables) for deeper analysis.

10 JUNIOR ENGINEER TIP



Do not rely only on traceroute to declare a network issue. Use it as a guide, but always test the actual application (e.g. curl) and collect more evidence before making changes.

“TRACE THE PATH.
FIND THE EVIDENCE.
SOLVE THE PROBLEM.”

PACKET CAPTURE WITH tcpdump

See the traffic. Find the problem. Prove the truth.

1 WHY PACKET CAPTURE MATTERS

- ✓ Shows the actual network traffic (what is really happening).
- ✓ Helps you confirm if packets reach a server.
- ✓ Useful when logs are not enough.
- ✓ Helps identify network, firewall, and application issues.
- ✓ Removes guesswork from troubleshooting.

3 TCPDUMP SYNTAX

```
tcpdump [options] [filter expression]
```

5 COMMON TCPDUMP COMMANDS

COMMAND	DESCRIPTION
<code>tcpdump -i eth0</code>	Capture on a specific interface (eth0).
<code>tcpdump -i any</code>	Capture on all interfaces.
<code>tcpdump host 192.168.1.10</code>	Capture traffic to or from a specific host.
<code>tcpdump port 80</code>	Capture traffic on a specific port.
<code>tcpdump tcp port 443</code>	Capture only TCP traffic on port 443.
<code>tcpdump udp port 53</code>	Capture only UDP traffic on port 53 (DNS).
<code>tcpdump icmp</code>	Capture only ICMP traffic.
<code>tcpdump -i eth0 host example.com</code>	Capture traffic to or from a hostname.
<code>tcpdump -i eth0 -nn</code>	Do not resolve names (faster output).
<code>tcpdump -i eth0 -w capture.pcap</code>	Save capture to a file.
<code>tcpdump -r capture.pcap</code>	Read and analyze a saved capture file.

8 PROVING PACKETS REACH A SERVER

- ✓ Run tcpdump on the destination server.
- ✓ Filter by the expected port or host.
- ✓ If you see the packets, the network is working and the issue is likely at the application layer.
- ✓ If you do not see the packets, the problem is before the server (e.g. routing, firewall, load balancer).

9 CAPTURE ONLY WHAT YOU NEED



- ✓ Use specific filters (host, port, protocol).
- ✓ Avoid capturing everything (can generate huge files).
- ✓ Capture for a short time, reproduce the issue, then stop.
- ✓ Save to a file if you need to share or analyze later.



JUNIOR ENGINEER TIP

Use packet capture as a diagnostic tool, not as a permanent solution. It helps you find the truth faster.

2 COMMON USE CASES

TROUBLESHOOT CONNECTIVITY ISSUES	VERIFY IF PACKETS REACH A SERVER	ANALYZE PROTOCOL BEHAVIOR	DEBUG APPLICATION TRAFFIC	INVESTIGATE SECURITY INCIDENTS

4 IMPORTANT CONCEPTS

INTERFACES	HOSTS	PORTS	PROTOCOL FILTERS
Use -i to specify the network interface.	Filter traffic to or from a specific host (IP or name).	Filter traffic using port numbers.	Filter by protocol (e.g. tcp, udp, icmp, http, https).

6 READING TCP PACKETS (SYN, ACK, RST)

FLAG	MEANING	WHAT IT TELLS YOU
SYN	Synchronize	Client is trying to start a connection.
SYN, ACK	Synchronize + Acknowledge	Server is responding.
ACK	Acknowledge	Connection is established.
RST	Reset	Connection is rejected (or closed).

7 EXAMPLE OUTPUT (REAL TRAFFIC)

```
$ tcpdump -nn -i eth0 port 80
12:01:23.123456 IP 192.168.1.10.54321 > 93.184.216.34.80:
S 1234567890:1234567890(0) win 64240
12:01:23.234567 IP 93.184.216.34.80 > 192.168.1.10.54321:
S. 987654321:987654321(0) ack 1234567891 win 65160
12:01:23.234890 IP 192.168.1.10.54321 > 93.184.216.34.80:
.. ack 987654322 win 64240
```

10 SECURITY AND PRIVACY AWARENESS



- ✓ Packet captures can contain sensitive data (usernames, passwords, tokens, personal information).
- ✓ Do not share captures publicly.
- ✓ Obfuscate or sanitize sensitive information.
- ✓ Follow your organization's security policies.
- ✓ Delete captures when no longer needed.



**PACKETS DON'T LIE.
OBSERVE. ANALYZE. FIX.**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
PRACTICE
BUILD
GROW



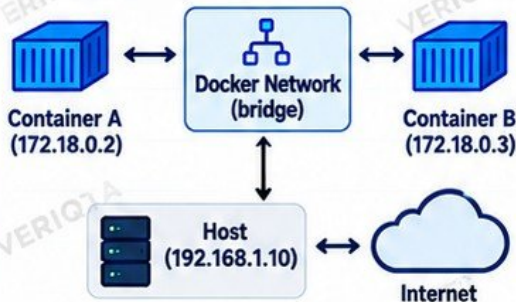
DOCKER AND CONTAINER NETWORK TROUBLESHOOTING

CONTAINERS
NETWORKS
REAL-WORLD
SOLUTIONS

Understand container networking. Find the issue. Keep your applications running.

1 CONTAINER NETWORKING MENTAL MODEL

Containers use virtual networks to communicate with each other, the host, and external networks.



2 CONTAINER IP vs HOST IP

- Each container gets its own IP address on the Docker network.
- Container IP is different from the host IP.
- Containers can communicate using container IPs or container names (on the same network).
- To access a container from outside, you must publish a port to the host.

Example

Container IP: 172.18.0.2
Host IP: 192.168.1.10
Published Port: 8080 (host) → 80 (container)

3 PORT PUBLISHING

- Exposes a container port to the host.
- Format: `-p <host_port>:<container_port>`
- Example:
`docker run -d -p 8080:80 nginx`
- Now you can access the container at:
`http://<host-ip>:8080`



Common Issues

- Wrong host port
- Port already in use
- Application not listening on the expected port
- Firewall blocking the port

4 USEFUL DOCKER COMMANDS

List networks

```
docker network ls
```

Shows all Docker networks.

Inspect a network

```
docker network inspect <network_name>
```

Shows detailed network configuration.

Inspect a container

```
docker inspect <container_name>
```

Shows container details (IP, networks, ports, etc.).

5 CONTAINER-TO-CONTAINER COMMUNICATION

- Containers on the same network can communicate using container names or IP addresses.
- Example: container A can reach container B using `http://<container_name>:<port>`



6 DNS BETWEEN CONTAINERS

- Docker provides internal DNS.
- Containers can resolve each other by container name.
- Example: if a container is named `api`, use `http://api:8080` from another container.
- Works on user-defined networks (bridge, overlay, etc.).

Test DNS

```
docker exec -it <container> nslookup api
```

Or use: `ping api`

7 WRONG PUBLISHED PORT

- The container may be running, but the wrong port is published to the host.
- Check which ports are actually exposed:
`docker ps`
Look at the PORTS column.
- Fix by using the correct `-p` mapping.

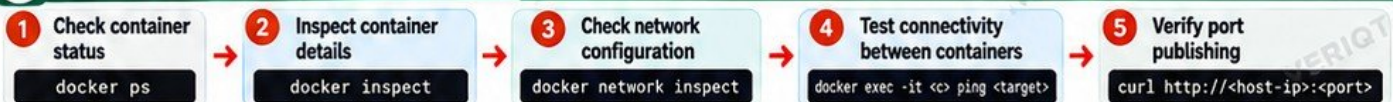
8 APPLICATION BOUND TO LOCALHOST

- If an application is bound to 127.0.0.1 inside the container, it will not be accessible from outside the container.
- Example: Nginx, Node.js, or other apps bound to localhost.
- Fix: configure the application to listen on 0.0.0.0 (all interfaces).

9 "WORKS INSIDE THE CONTAINER" IS USEFUL EVIDENCE

- If the application works inside the container (e.g. `curl localhost:8080`), it means:
 - ✓ The application is running.
 - ✓ The issue is likely network-related (port publishing, firewall, host network, etc).
- It helps you narrow down the problem faster.

10 COMMON TROUBLESHOOTING FLOW



11 JUNIOR ENGINEER TIP



"Works inside the container" is valuable evidence. It means the application is running, so focus on networking, port publishing, firewall rules, or configuration issues.

“UNDERSTAND.
TROUBLESHOOT.
KEEP BUILDING.”



KUBERNETES NETWORK TROUBLESHOOTING

Find the issue. Understand the flow. Keep your cluster running.

PODS
SERVICES
DNS
NETWORKPOLICY
REAL-WORLD
SOLUTIONS

1 POD NETWORKING

- Each Pod gets its own IP address.
- Pods can communicate with each other directly (within the same cluster).
- The cluster network (CNI) handles Pod IP allocation and routing.
- Pods are ephemeral, so IPs can change when a Pod is recreated.

2 POD IPs

- Each Pod has a unique IP in the cluster.
- Check Pod IPs with:
`kubectl get pods -o wide`
- Pod IPs are used for Pod-to-Pod communication.
- If a Pod cannot be reached, check if it is running and has a valid IP.

3 SERVICES

- A Service provides a stable endpoint for a set of Pods.
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName.
- Services use labels to find Pods.
- If the Service is not working, check endpoints, selectors, and Pod health.

4 ClusterIP

- Default Service type.
- Only accessible inside the cluster.
- Provides a virtual IP (ClusterIP) that load balances to Pods.
- Check with:
`kubectl get svc`
- Look at the CLUSTER-IP column.

5 Endpoints and EndpointSlices

- Endpoints show the actual Pod IPs behind a Service.
- EndpointSlices are the newer, scalable version of Endpoints.
- If a Service has no endpoints, traffic will not reach any Pod.
- Check with:
`kubectl get endpoints`
or `kubectl get endpointslices`

6 CoreDNS

- Provides DNS for the cluster.
- Pods use CoreDNS to resolve Service names (e.g. `myapp.default.svc.cluster.local`).
- If DNS is not working, Pods cannot reach Services by name.
- Check CoreDNS Pods:
`kubectl get pods -n kube-system | grep coredns`
- Check DNS from a Pod:
`kubectl exec -it <pod> -- nslookup <service>`

7 NetworkPolicy

- Controls traffic between Pods, namespaces, and external sources.
- Can allow or deny traffic based on labels, namespaces, ports, and protocols.
- If Pods cannot communicate, check if a NetworkPolicy is blocking traffic.
- Check with:
`kubectl get networkpolicy -A`

8 USEFUL KUBECTL COMMANDS

List Pods (with IPs)	<code>kubectl get pods -o wide</code>
List Services	<code>kubectl get svc</code>
List Endpoints	<code>kubectl get endpoints</code>
Describe a Pod	<code>kubectl describe pod <pod></code>
Describe a Service	<code>kubectl describe svc <svc></code>
Describe Endpoints	<code>kubectl describe endpoints <svc></code>
Check NetworkPolicy	<code>kubectl get networkpolicy -A</code>
View logs	<code>kubectl logs <pod></code>
Exec into a Pod	<code>kubectl exec -it <pod> -- sh</code>

9 TESTING CONNECTIVITY

- Test Pod-to-Pod communication:
`kubectl exec -it <pod1> -- ping <pod2-ip>`
- Test Pod-to-Service communication:
`kubectl exec -it <pod> -- curl <service-name>`
- Test DNS resolution:
`kubectl exec -it <pod> -- nslookup <service>`
- Test from a temporary Pod:
`kubectl run test --rm -it --image=busybox -- sh`

10 SEPARATING FAILURES

LAYER	WHAT TO CHECK	EXAMPLE COMMAND	COMMON ISSUE
Application	Is the app running?	<code>kubectl logs <pod></code>	App error, crash, wrong config
Service	Does it have endpoints?	<code>kubectl get endpoints</code>	No endpoints, wrong selector
DNS	Can the name be resolved?	<code>nslookup <service></code>	CoreDNS issue, wrong name
Network	Is traffic allowed?	<code>kubectl get networkpolicy</code>	NetworkPolicy blocking traffic

12 JUNIOR ENGINEER TIP



"Works inside the Pod" is valuable evidence. It means the application is likely running, so the issue is in the Service, DNS, network, or outside the cluster.

11 EXAMPLE: TROUBLESHOOTING FLOW

- Check if the Pod is running (`kubectl get pods -o wide`)
- Check Service and endpoints (`kubectl get svc`, `get endpoints`)
- Test DNS resolution (`nslookup <service>`)
- Test connectivity (ping, curl)
- Check NetworkPolicy (`kubectl get networkpolicy`)
- Check application logs (`kubectl logs`)
- Identify the failing layer and fix it

**CHECK EACH LAYER.
FIND THE BREAK.
FIX WITH EVIDENCE.**



LOAD BALANCER AND REVERSE PROXY FAILURES



Follow the traffic. Find the failure. Keep your services available.

1 ARCHITECTURE OVERVIEW

Client → Load Balancer → Backend



2 HEALTHY vs UNHEALTHY TARGETS

- Load balancer distributes traffic only to healthy targets.
- Targets are marked unhealthy if they fail health checks.
- Common reasons:
 - Application not running
 - Wrong port
 - Slow responses
 - High error rate
 - Network or firewall issues



3 HEALTH CHECKS

- Load balancer uses health checks to monitor target health.
- Checks a specific path, port and response code.
- If health checks fail, the target is marked unhealthy.

Example (HTTP health check)

Path: /health
Port: 80
Expected status: 200 OK
Interval: 30 seconds
Timeout: 5 seconds
Unhealthy threshold: 2

4 WRONG BACKEND PORTS

- Traffic reaches the target but the application is not listening on the expected port.
- Common in containers and Kubernetes when the Service or target port is misconfigured.
- Always verify the actual port the application is listening on.

Check listening ports:

```
ss -ltnp | grep <port>
or inside a container:
netstat -tlnp
```

5 SECURITY RULES

- Ensure security groups, firewalls, or Network ACLs allow traffic:
 - From client → load balancer
 - Load balancer → backend targets
 - Required ports (e.g. 80, 443, 8080)
- Check both inbound and outbound rules.

If the load balancer cannot reach the target due to a security rule, the target will appear unhealthy.

6 TLS TERMINATION

- TLS can be terminated at the load balancer or at the reverse proxy (e.g. NGINX).
- If TLS is terminated at the load balancer, traffic to the backend may be HTTP.
- If TLS is terminated at NGINX, ensure the certificate is valid and correctly configured.
- Common issues:
 - Expired certificate
 - Wrong domain name (SNI)
 - Incomplete certificate chain
 - TLS version or cipher mismatch



7 NGINX / REVERSE PROXY AWARENESS

- NGINX is commonly used as a reverse proxy in front of applications.
- It receives client requests and forwards them to backend services.
- Common issues:
 - Wrong upstream server/port
 - Misconfigured proxy_pass
 - Timeout settings
 - Incorrect headers (e.g. X-Forwarded-For)
 - NGINX returning 502, 503 or 504 errors



Test NGINX configuration:

```
nginx -t
```

Check NGINX logs:

```
tail -f /var/log/nginx/error.log
```

8 COMMON ERROR CODES

502 Bad Gateway

- Load balancer or proxy received an invalid response from backend.
- Usually backend is down, wrong port, or misconfiguration.

503 Service Unavailable

- No healthy targets available.
- Application is down or failing health checks.
- Can also be due to maintenance or overload.

504 Gateway Timeout

- The proxy or load balancer did not receive a response in time.
- Backend is slow or not responding.
- Check backend performance and network connectivity.

9 HOW TO TEST EACH HOP INDEPENDENTLY



- Test from client
`curl -v https://your-domain.com`
- Test load balancer
`curl -v http://<lb-ip-or-dns>`
- Test backend target directly
`curl -v http://<target-ip>:<port>`
- Test NGINX (if used)
`curl -v http://<nginx-ip>`
- Test the application
`curl -v http://<pod-ip>:<port>/health`

10 JUNIOR ENGINEER TIP



Test each hop independently to identify where the failure is. Do not assume the problem is the application — it could be DNS, the load balancer, security rules, health checks, or the reverse proxy.

“ISOLATE THE FAILURE. TEST EACH LAYER. FIX WITH EVIDENCE.”



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
PRACTICE
BUILD
GROW

CLOUD NETWORK TROUBLESHOOTING

From VPC/VNet Design to Real-World Network Fixes

1 VPC/VNet THINKING

- A VPC (AWS) or VNet (Azure) is your isolated cloud network.
- It defines IP ranges, subnets, route tables and network controls.
- Think in layers: subnets → routing → security → internet access.
- Most connectivity issues are caused by a missing or incorrect configuration in one of these layers.

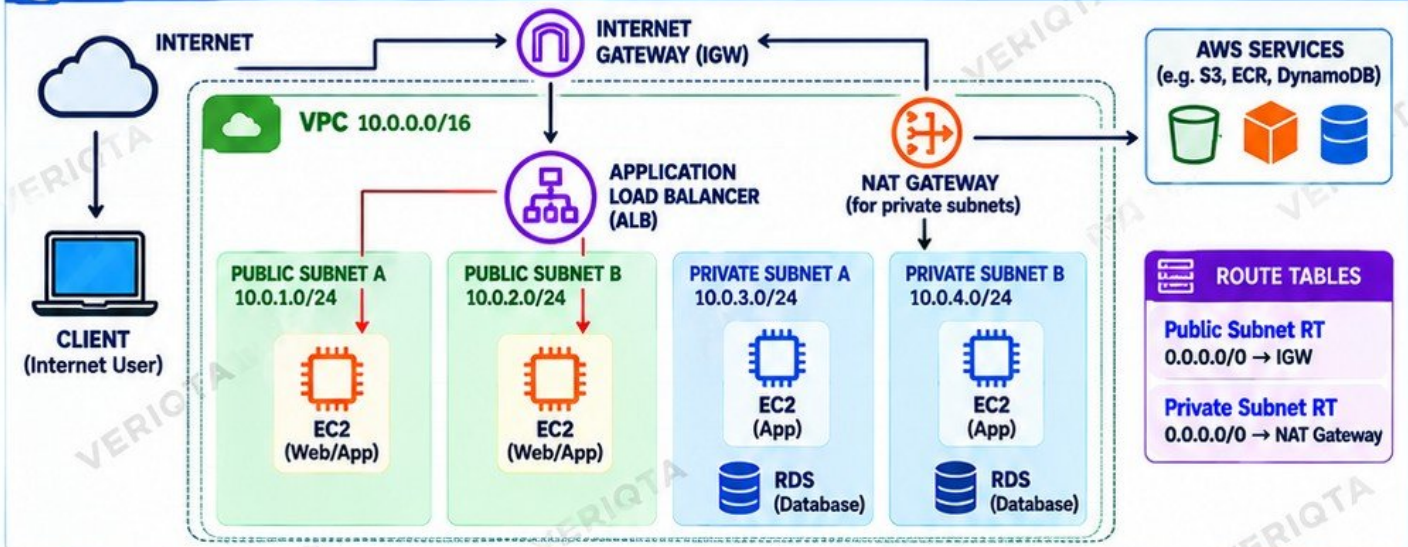
2 KEY COMPONENTS

- VPC/VNet**
Isolated network in the cloud.
- Subnets**
Public and private subnets.
- Route Tables**
Decide where traffic goes.
- Internet Gateway (IGW)**
Provides internet access for public subnets.
- NAT Gateway**
Allows private subnets to access the internet (outbound only).

3 SECURITY CONTROLS

- Security Groups**
Stateful firewall at the instance or resource level (allow/deny traffic).
- Network ACLs**
Stateless firewall at the subnet level (allow/deny traffic).
- Public IPs**
Required for direct internet access (e.g. EC2, Load Balancer).
- Private Workloads**
Resources in private subnets (without public IPs).
- Load Balancers**
Distribute traffic to multiple instances (ALB/NLB).

4 TYPICAL CLOUD NETWORK ARCHITECTURE (AWS EXAMPLE)



5 COMMON ISSUES AND FIXES

- EC2 HAS NO INTERNET**

 - No public IP (for public subnet)
 - Missing/incorrect route to IGW
 - Security Group blocking outbound traffic
 - NACL blocking traffic
 - Instance in private subnet without NAT Gateway
- PUBLIC APPLICATION CANNOT BE REACHED**

 - No public IP or Load Balancer
 - Incorrect Security Group rules
 - NACL blocking inbound traffic
 - Wrong route table configuration
 - Application not listening on the correct port
 - DNS pointing to the wrong IP

7 JUNIOR ENGINEER TIP

Troubleshoot systematically. Check each layer, collect evidence, and verify the real cause before making changes. Cloud networking issues are usually configuration problems, not random failures.

6 FOLLOW THE CLOUD TRAFFIC PATH

- 1 Start from the client (DNS, IP, connectivity).
- 2 Check the Load Balancer (is it reachable?).
- 3 Verify the subnet (public or private).
- 4 Check the route table (where does it go?).
- 5 Inspect Security Groups and NACLs.
- 6 Confirm instance configuration (public IP, service).
- 7 Test end-to-end (client → load balancer → instance).

**FOLLOW THE TRAFFIC.
FIND THE MISCONFIGURATION.
FIX IT. VERIFY IT.**

PRODUCTION NETWORK INCIDENT ISOLATE THE ROOT CAUSE

A systematic approach to diagnose, fix, and learn from real network incidents.

1 SCENARIO

Application suddenly becomes unreachable

- Users report they cannot access the application.
- It was working before.
- You need to find the root cause and restore service quickly and safely.

2 CONFIRM THE SCOPE



- Is it all users or a specific group?
- Is it one service or multiple services?
- Is it a regional or global issue?
- Internal or external access?
- Check error messages and time of impact.

Example questions:

- Does it work from inside the VPC?
- Does it work from a different network?
- Is it a new or existing issue?

3 CHECK RECENT CHANGES



- Deployments (app, infra, network)
- Security group or firewall changes
- DNS changes
- Load balancer changes
- Kubernetes changes
- Cloud route table or subnet changes
- Certificate or DNS record updates
- Any maintenance or ongoing work

Look at:

- Change management system
- Git commits / CI/CD logs
- Cloud audit logs (e.g. AWS CloudTrail)
- Kubernetes events

4 TEST DNS



- Does the domain resolve?
- Is it the correct IP?
- Are there DNS errors?

```
dig example.com
nslookup example.com
getent hosts example.com
```

5 TEST ENDPOINT



- Can you reach the service?
- What error do you get?
- Is it timeout, refused or HTTP error?

```
curl -v https://example.com
curl -I https://example.com
ping example.com
```

6 INSPECT PORTS



- Is the port open and listening?
- Is the service running?

```
ss -lntp
ss -lunp
nc -vz example.com 80
nc -vz example.com 443
```

7 INSPECT ROUTES



- Is the traffic going to the right destination?
- Is the route table correct?
- Is the gateway available?

```
ip route
ip route get <IP>
traceroute <IP>
tracert <IP>
```

8 INSPECT FIREWALL RULES



- Check security groups (cloud)
- Check network ACLs
- Check host firewall (iptables/nftables)
- Verify inbound and outbound rules
- Confirm source and destination

9 CHECK LOAD BALANCER HEALTH



- Is the load balancer healthy?
- Are targets healthy?
- Are health checks passing?
- Is traffic reaching the backend?
- Check listener and target group configuration.

10 INSPECT KUBERNETES RESOURCES



- Check Pods, Services, Endpoints
- Look for crashes or restarts
- Check Events and logs
- Verify NetworkPolicy rules
- Ensure the correct Pods are behind the Service

```
kubectl get pods,svc,ep -n <ns>
kubectl describe <resource>
kubectl logs <pod>
kubectl get networkpolicy -n <ns>
```

11 CAPTURE PACKETS WHEN NECESSARY



- Use packet capture to see what is actually happening.
- Confirm if packets are reaching the server.
- Identify resets, timeouts or dropped packets.

```
tcpdump -i any host <IP>
tcpdump -i any port 80
tcpdump -nn -vv
```

12 IDENTIFY ROOT CAUSE



- Use the evidence to find the exact cause.
- Common causes: DNS, firewall, wrong route, service down, misconfiguration, bad deployment.
- Confirm the root cause with multiple tests.

13 APPLY THE SMALLEST SAFE FIX



- Make only the necessary change.
- Avoid broad or risky changes (e.g. opening all ports).
- Test the fix in a controlled way.
- Document what you changed.

14 VERIFY RECOVERY



- Confirm the application is reachable.
- Test from different locations.
- Check monitoring and logs.
- Ensure there are no side effects.

15 RECORD EVIDENCE FOR THE INCIDENT REVIEW



- Document the timeline (what happened, when).
- Record commands, outputs and screenshots.
- Write the root cause, the fix, and lessons learned.
- Update runbooks to prevent or reduce future occurrences.

“**REAL ENGINEERS FIND THE CAUSE, NOT JUST A TEMPORARY FIX.**”

Same Incidents.
Stronger Engineers.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
PRACTICE
BUILD
GROW

FINAL TROUBLESHOOTING LAB

FOLLOW THE PACKET

A hands-on, end-to-end lab to diagnose, fix, and recover a real-world network issue.

1 STEP-BY-STEP LAB GUIDE

- 1  **Production-style failure scenario**
A realistic, real-world scenario.
- 2  **User reports application unavailable**
Confirm the exact symptom.
- 3  **Verify DNS**
Check if the domain resolves.
- 4  **Resolve destination IP**
Find the correct IP address.
- 5  **Test connectivity**
Check basic network reachability.
- 6  **Trace the route**
Identify where traffic stops.
- 7  **Test TCP port**
Verify the port is open.
- 8  **Inspect TLS**
Check certificate and handshake.
- 9  **Test HTTP**
Validate the application response.
- 10  **Inspect load balancer**
Check health and target status.
- 11  **Inspect Kubernetes Service**
Verify service configuration.
- 12  **Inspect endpoints**
Check endpoint availability.
- 13  **Inspect Pods**
Ensure pods are running and healthy.
- 14  **Check NetworkPolicy / firewall controls**
Look for blocked traffic.
- 15  **Capture packets**
Use tcpdump to see what is happening.
- 16  **Find the deliberately introduced fault**
Identify the root cause.
- 17  **Fix the root cause**
Apply the smallest safe fix.
- 18  **Verify end-to-end recovery**
Confirm the application is working.

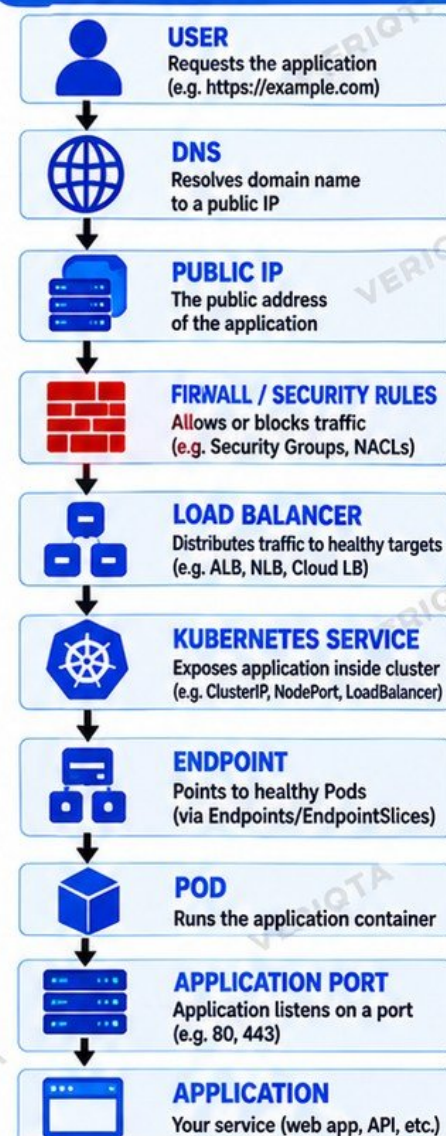
2 USEFUL COMMANDS

DNS lookup	<code>dig example.com</code> <code>nslookup example.com</code>
Resolve IP	<code>getent hosts example.com</code>
Test connectivity	<code>ping <IP></code>
Trace route	<code>traceroute <IP></code> <code>tracert <IP></code>
Test port	<code>nc -vz <IP> <port></code> <code>telnet <IP> <port></code>
Test TLS	<code>openssl s_client -connect <IP>:443 -servername example.com</code>
Test HTTP	<code>curl -v https://example.com</code> <code>curl -I https://example.com</code>
Check load balancer (targets/health)	<code>aws elbv2 describe-target-health</code> # or cloud provider CLI
Check Kubernetes resources	<code>kubectl get svc -A</code> <code>kubectl get endpoints -A</code> <code>kubectl get pods -A</code>
Check NetworkPolicy	<code>kubectl get networkpolicy -A</code> <code>kubectl describe networkpolicy <name></code>
Capture packets	<code>tcpdump -i any host <IP></code> <code>tcpdump -nn -vv port <port></code>

3 FIND AND FIX THE FAULT

- ✓ Analyze the evidence from each step.
- ✓ Compare expected vs actual behavior.
- ✓ Identify the layer where the failure occurs.
- ✓ Apply the smallest safe fix.
- ✓ Test again after the fix.

4 FINAL TROUBLESHOOTING ARCHITECTURE



5 SCENARIO EXAMPLE

- Users cannot access `https://app.example.com`
- The application was working earlier.
- A recent change was made to the infrastructure.
- Your task: find the issue and restore access.

6 SUCCESS CRITERIA

- ✓ Identify the exact root cause.
- ✓ Apply the correct fix.
- ✓ Verify end-to-end recovery.
- ✓ Document your findings.

7 JUNIOR ENGINEER TIP

“Follow the packet, follow the evidence, not assumptions.”



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
PRACTICE
BUILD
GROW